

1) Requerimientos vs. especificaciones

Requerimientos

1. El sistema debe permitir consultar prácticas por tipo y por año para facilitar la exploración de ofertas históricas.
2. El sistema debe mostrar información institucional en módulos de noticias, proyectos, becados, workshops y giras.
3. El sistema debe permitir autenticación de administradores para proteger la gestión de contenido.
4. El sistema debe permitir crear, editar y eliminar contenido desde un panel administrativo.

Especificaciones

1. La consulta de prácticas se expone mediante endpoints REST en */api/practicas/[type]* y */api/practicas/[type]/years*
2. Los módulos públicos se consumen desde endpoints dedicados (*/api/noticias*, */api/proyectos*, */api/becados*, */api/workshops*, */api/gira*) y se renderizan en ViteJs.
3. La autenticación administrativa utiliza rutas en */api/admin/auth/** con soporte de login, refresh y logout.
4. La persistencia principal de datos administrativos se implementa sobre MongoDB con Prisma como capa de acceso.

2) Requisitos funcionales y no funcionales

Requisitos funcionales

1. El usuario final puede visualizar listados y detalle de prácticas.
2. El administrador autenticado puede gestionar (alta, edición, eliminación) noticias, proyectos, becados, talleres y giras.

Requisitos no funcionales

1. Seguridad: sólo usuarios autenticados pueden operar endpoints de administración.
2. Mantenibilidad: separación por capas (routes/controllers/services/models) para facilitar evolución y pruebas.

3) Modelo arquitectónico

Sí, en la documentación y estructura del repositorio se identifica un modelo arquitectónico **cliente-servidor por capas**:

- Capa de presentación: `Frontend` (portal público) y `FrontendAdmin` (panel administrativo), ambos en React + Vite.
- Capa de API / aplicación: `Backend/apis` (Next.js API routes) con separación en controladores, servicios y modelos.
- Capa de datos: MongoDB (y soporte adicional de Elastic para consultas específicas).
- Integración externa: synchronizer en Python para ingesta desde Google Sheets hacia la API.

Además el **backend** sigue un patrón de diseño **MVC + Service Layer**:

- Rutes: si bien no se encuentra estrictamente presente, NEXT.js utiliza la estructura de carpetas como rutas.
- Controller: recibe la solicitud HTTP, valida y coordina el flujo.
- Service: concentra reglas de negocio y casos de uso.
- Model: encapsula acceso a datos/persistencia.